DOCUMENTO DE ARQUITECTURA (ARD)

Norvex

Arquitectura, decisiones técnicas y requerimientos de calidad

Materia	Metodologías Ágiles
Equipo	Grupo 2
Versión	1.0 — proyecto completo
Fecha	12/11/2026
Sistema publicado	https://norvex.onrender.com
Código documentado	rama `main` @ `4a388aa`
Documento relacionado	PRD — Documento de Requerimientos de Producto

Índice

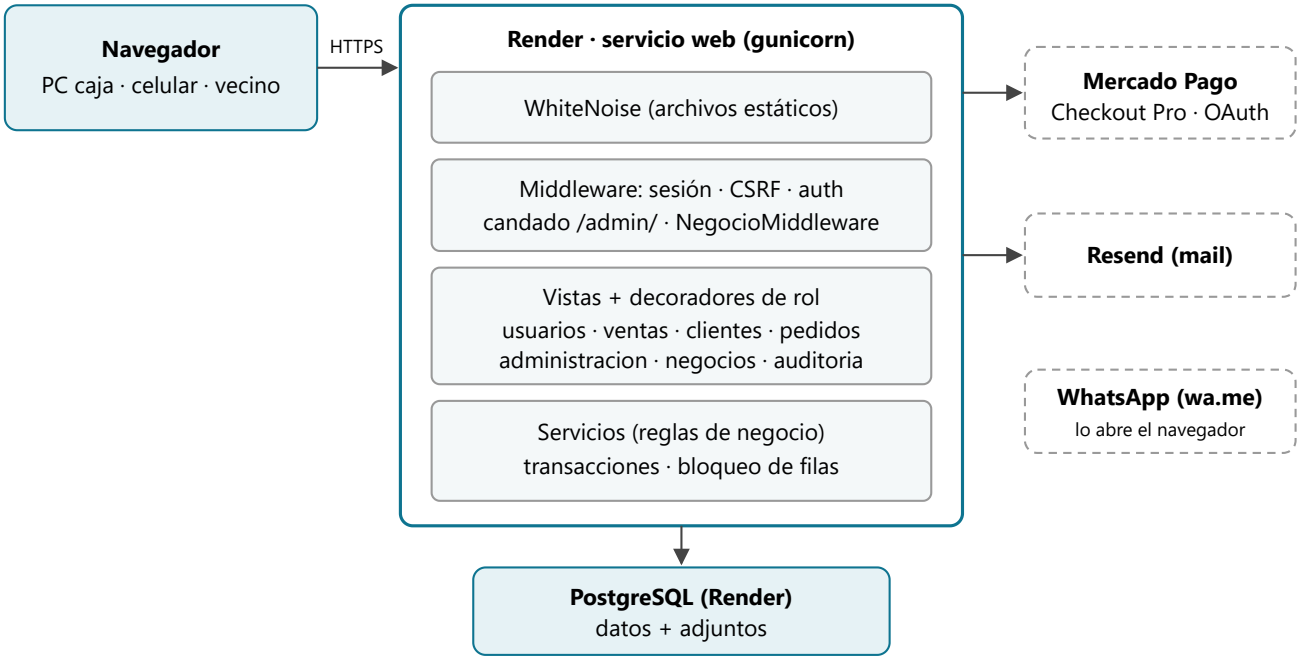
1. Propósito y alcance
2. Vista general
3. Stack tecnológico
4. Módulos del sistema
5. Arquitectura multiempresa
6. Modelo de datos
7. Seguridad y autorización
8. Flujos técnicos clave
9. Integraciones externas
10. Despliegue y configuración
11. Decisiones de arquitectura
12. Calidad y pruebas
13. Riesgos técnicos y evolución

1. Propósito y alcance

Este documento describe **cómo** está construido Norvex: componentes, datos, seguridad, integraciones, despliegue y las decisiones técnicas que se tomaron. El **qué** (requerimientos, roles e historias de usuario) está en el PRD. Cubre el proyecto completo, desde el Sprint 1 hasta la versión publicada.

2. Vista general

Norvex es un **monolito web con renderizado en el servidor**. Django arma las páginas HTML y las envía al navegador. Las partes interactivas (búsqueda, carrito, estado del QR) usan **HTMX** y JavaScript liviano, sin un frontend aparte ni una API REST pública. Toda la información vive en una base **PostgreSQL**.



3. Stack tecnológico

Capa	Tecnología	Para qué
Lenguaje	Python 3.13	Todo el backend
Framework	Django 6.1.1	ORM, vistas, plantillas, autenticación, formularios, migraciones, admin
Base de datos	PostgreSQL (psycopg2) · SQLite en desarrollo	Persistencia; SQLite solo como respaldo local si no hay `DATABASE_URL`
Frontend	Plantillas de Django, Bootstrap 5.3, Bootstrap Icons, HTMX 1.9, CSS y JS propios	Interfaz responsive sin paso de compilación
Servidor	gunicorn + WhiteNoise	Servir la app y los estáticos en producción
Configuración	django-environ	Leer variables de entorno y `.env`
Integraciones	requests (Mercado Pago), resend (mail)	Pagos y recuperación de contraseña
Exportación	openpyxl	Reporte de stock en Excel
Hosting	Render (servicio web + PostgreSQL)	Publicación en la nube
Control de versiones	Git + GitHub	Trabajo en equipo; cada push a `main` se publica solo

> La base empezó en SQLite (la que trae Django), pasó a PostgreSQL en Supabase y terminó en PostgreSQL de Render, en el mismo proveedor que la aplicación para bajar la latencia.

4. Módulos del sistema

El proyecto se divide en apps de Django por dominio. Cada app tiene sus modelos, vistas, URLs, plantillas y tests. Las reglas de negocio importantes están en archivos `services.py`, separadas de las vistas.

App	Responsabilidad	Piezas principales
`usuarios`	Login, registro, roles, recuperación de contraseña, empleados	`Empleado`, decoradores `rol_requerido` y `superadmin_requerido`, middlewares, backend de mail
`negocios`	Multiempresa, landing, facturas, proveedores, adjuntos	`Negocio`, `Proveedor`, `Documento`, `PagoDocumento`, `ArchivoGuardado`, storage en base
`ventas`	Catálogo, caja, turnos, arqueo, Mercado Pago	`Producto`, `Venta`, `Turno`, `ArqueoTurno`, `PagoMercadoPago`, `services.py`, `cobro_qr.py`, `mercadopago.py`
`clientes`	Clientes frecuentes, puntos y cuenta corriente	`Cliente`, `PagoCuentaCorriente`, validación del límite de fiado
`pedidos`	Catálogo público, pedidos, reservas, entrega	`Pedido`, `ItemPedido`, `ReservaStock`, mensaje de WhatsApp

`administracion`	Panel, gestión de usuarios, reportes, superadmin	`reportes.py` (consultas puras), exportación a Excel
`auditoria`	Registro de acciones	`RegistroAuditoria`, señales de login/logout, `registrar_accion`
`config`	Configuración del proyecto	`settings.py`, `urls.py`, `wsgi.py`

5. Arquitectura multiempresa

Se eligió el modelo de **base compartida con columna de negocio**: todos los negocios usan las mismas tablas, y cada registro operativo tiene una clave foránea a `Negocio`.

- **Resolución del negocio:** `NegocioMiddleware` deja `request.negocio` en cada petición. En la tienda pública (`/t/<slug>/`) lo saca de la URL; en el resto, del `Empleado` del usuario logueado. El superadmin no tiene negocio y ve todo.
- **Aislamiento:** cada vista filtra sus consultas por `request.negocio`. Nunca se confía en un id de negocio que mande el navegador.
- **Pertenencia:** `Empleado` une un usuario con un único negocio (uno a uno). El rol (Administrador o Cajero) es un grupo de Django, independiente del negocio.
- **Migración:** el sistema nació para un solo kiosco. Los comandos `migrar_a_multiempresa` y `backfill_negocios` asignaron los datos existentes al negocio "Kiosco De Barrio" sin perder información.

6. Modelo de datos

6.1 Entidades

Entidad	Campos principales	Relaciones
Negocio	nombre, slug único, dirección, teléfono, WhatsApp, email, horarios, logo, color, activo	Raíz de todos los datos de un comercio
Empleado	activo	1:1 con User; N:1 con Negocio
Categoria	nombre	N:1 con Negocio
Producto	nombre, código de barras único, precio, costo, stock, stock mínimo, activo	N:1 con Categoria y Negocio
Venta	fecha, total, método de pago, monto entregado, vuelto, puntos ganados, finalizada	N:1 con User (cajero), Cliente y Negocio
DetalleVenta	cantidad, precio unitario y costo unitario congelados	N:1 con Venta y Producto
Turno	apertura, cierre, totales de efectivo, tarjeta y Mercado Pago, deuda otorgada, operaciones	N:1 con User y Negocio
ArqueoTurno	montos del sistema, montos físicos y diferencias por medio de pago, observaciones	N:1 con Turno y User

PagoMercadoPago	token, id de preferencia, link de pago, simulado, estado, id de pago	1:1 con Venta
ConexionMercadoPago	access token, refresh token, vencimiento, cuenta, producción	N:1 con User y Negocio
Cliente	nombre, apellido, DNI único, teléfono, email, puntos, límite de crédito, saldo de deuda, activo	Tiene ventas y pagos
PagoCuentaCorriente	monto, fecha, observación	N:1 con Cliente, Negocio y User
Pedido	nombre y teléfono del cliente, comentario, tipo de entrega, costo de envío, estado, total	N:1 con Negocio y Cliente; N:1 con la Venta que lo cobró
ItemPedido	nombre, cantidad, precio unitario congelado	N:1 con Pedido y Producto
ReservaStock	cantidad, fecha de reserva, vencimiento (48 h), estado	N:1 con Producto, Pedido y Cliente
Proveedor	nombre, teléfono, email, activo	N:1 con Negocio
Documento	categoría, número, monto, monto pagado, emisión, vencimiento, estado, motivo de anulación, archivo	N:1 con Negocio y Proveedor
PagoDocumento	fecha, monto, método, comprobante, grupo de pago	N:1 con Documento, Negocio y User
ArchivoGuardado	nombre único, contenido binario, tipo	Guarda los adjuntos (ver §10)
RegistroAuditoria	acción, detalle, fecha, IP	N:1 con User

6.2 Reglas de integridad

- **Precios congelados:** cada línea de venta y de pedido guarda el precio (y la venta también el costo) del momento. Cambiar un precio no modifica ventas pasadas ni pedidos ya hechos.
- **Bajas lógicas:** productos y clientes se desactivan con `activo=False` en lugar de borrarse.
- **Totales congelados:** al cerrar el turno, los totales se copian al `Turno` y no cambian aunque después se registren otras ventas.
- **Stock disponible:** se calcula como `stock − reservas vigentes`. Una reserva vencida deja de contar sola, sin necesidad de una tarea programada.

7. Seguridad y autorización

Aspecto	Cómo se resuelve
Autenticación	Sistema de usuarios de Django; contraseñas con hash PBKDF2 y validadores de fortaleza
Roles	Grupos de Django `Administrador` y `Cajero`; el superadmin es `is_superuser`
Autorización	Decorador `@rol_requerido(...)` en cada vista; si el rol no corresponde, devuelve 403 desde el servidor
Aislamiento entre negocios	Filtro por `request.negocio` en cada consulta; un recurso de otro negocio responde 404

Panel de Django	Middleware de doble candado: 404 para quien no es administrador, y nueva confirmación de contraseña con vencimiento configurable
Formularios	Token CSRF en todos los POST; logout solo por POST
Producción	HTTPS detrás del proxy de Render, cookies `Secure`, `DEBUG=False`, `SECRET_KEY` generada por el hosting
Secretos	Credenciales en variables de entorno, nunca en el repositorio; las claves de los usuarios demo se cambian por migración guardando solo el hash
Pagos	El aviso (webhook) de Mercado Pago no se acepta a ciegas: el sistema vuelve a consultar el pago a la API antes de aprobar la venta
Adjuntos	Se sirven por una vista con login que verifica que el archivo sea del negocio del usuario
Trazabilidad	Auditoría con usuario, acción, fecha e IP (contempla proxies con `X-Forwarded-For`)

8. Flujos técnicos clave

8.1 Confirmar una venta

El carrito vive en la sesión del usuario. Al confirmar, `ventas/services.py` abre una transacción (`transaction.atomic`) y bloquea las filas de los productos con `select_for_update`. Dentro de esa transacción:

1. Valida el stock.
2. Si es fiado, valida el límite del cliente.
3. Crea la venta con sus líneas.
4. Descuenta el stock con expresiones `F()`.
5. Suma la deuda y los puntos.

Si algo falla, se deshace todo y el carrito queda igual. El bloqueo evita que dos cajas vendan la última unidad al mismo tiempo.

El mismo núcleo (`_crear_venta`) lo usan la caja, la entrega de pedidos y el cobro con QR. Las reglas de stock, fiado y puntos están en un solo lugar.

8.2 Cobro con QR de Mercado Pago

1. Se crea la venta con `finalizada=False`: el stock queda reservado, pero la venta no cuenta en reportes ni en el turno.
2. Se pide a Mercado Pago una preferencia de Checkout Pro con referencia `norvex-venta-<id>` y vencimiento de 30 minutos. El link se muestra como QR.
3. La confirmación llega por webhook o por consulta periódica desde la caja. En los dos casos se verifica el pago contra la API.
4. Aprobado: la venta pasa a finalizada y suma puntos. Cancelado: se devuelve el stock.
5. Sin credenciales, el link apunta a una página propia de pago simulado.

8.3 Pedido online y entrega

1. El carrito del visitante vive en la sesión.

2. Al confirmar se crean el `Pedido`, sus ítems con precio congelado y una `ReservaStock` por producto, que vence a las 48 horas.
3. Se arma un link `wa.me` con el mensaje para el WhatsApp del negocio.
4. Al entregar se bloquea el pedido y se genera la venta con el núcleo común. El pedido queda entregado y la reserva pasa a "convertida", todo en una transacción.

8.4 Cierre de turno y arqueo

`cerrar_turno` calcula los totales por medio de pago (lo fiado aparte), los guarda en el turno y registra la fecha de cierre, todo bajo bloqueo. Después, `registrar_arqueo` guarda los montos físicos contados y las diferencias contra los del sistema.

9. Integraciones externas

Servicio	Uso	Si no está configurado
Mercado Pago (Checkout Pro)	Crear el link de pago, consultar el estado, webhook	Pago simulado propio del sistema
Mercado Pago (OAuth)	Botón "Conectar con Mercado Pago": guarda y renueva el token de la cuenta del negocio	Se usa el token del entorno o el modo de prueba
Resend	Mail de recuperación de contraseña (HTML + texto)	El mail se imprime en la consola del servidor
WhatsApp	Link `wa.me` con el pedido armado; no requiere API	Se avisa que el negocio no cargó su número

La regla para todas las integraciones es la misma: **si falta una credencial, el sistema sigue funcionando** con un reemplazo de prueba y nunca devuelve un error 500.

10. Despliegue y configuración

10.1 Render

El archivo `render.yaml` describe la infraestructura: un servicio web Python y una base PostgreSQL (`norvex-db`), los dos en plan gratuito. Cada push a `main` dispara una publicación automática con estos pasos:

1. `pip install -r requirements.txt`
2. `python manage.py collectstatic`
3. `python manage.py migrate`
4. `python manage.py setup_inicial` (grupos y usuarios base; se puede repetir sin romper nada)
5. Arranque con `gunicorn config.wsgi:application`

Como el hosting no da una consola, los cambios de datos en producción (por ejemplo, rotar las claves de los usuarios demo o asignar un usuario a un negocio) se hacen con **migraciones de datos**, que corren solas en el paso `migrate`.

10.2 Archivos adjuntos

El disco del servicio web de Render es efímero: se borra en cada publicación y reinicio. Por eso los PDF de facturas y comprobantes se guardan en la base de datos, con un *storage* propio de Django (`negocios/storage.py`) y la tabla `ArchivoGuardado`. Se sirven por `/archivos/<nombre>`, previa verificación de permisos.

10.3 Variables de entorno

Variable	Uso
`DATABASE_URL`	Conexión a PostgreSQL (la inyecta Render)
`SECRET_KEY`, `DEBUG`	Seguridad de Django en producción
`ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`	Dominios permitidos; el de Render se agrega solo
`MP_ACCESS_TOKEN`, `MP_CLIENT_ID`, `MP_CLIENT_SECRET`, `MP_OAUTH_PRUEBA`	Mercado Pago
`RESEND_API_KEY`, `DEFAULT_FROM_EMAIL`	Envío de mails
`KIOSCO_NOMBRE`, `KIOSCO_WHATSAPP`	Datos del catálogo general

10.4 Datos de demostración

Los comandos `cargar_demo`, `cargar_demo_clientes` y `cargar_demo_todos` cargan los tres negocios con productos, ventas históricas, clientes y pedidos. Usan una semilla fija, así que se pueden correr de nuevo y dejan siempre los mismos datos.

11. Decisiones de arquitectura

#	Decisión	Motivo	Alternativa descartada
AD-01	Monolito Django con renderizado en servidor + HTMX	Un solo lenguaje y un solo despliegue; interactividad sin compilar un frontend	API REST + SPA (React)
AD-02	PostgreSQL	Concurrencia real con bloqueo de filas para varias cajas; recomendada por Django	SQLite (un solo archivo, bloquea toda la base al escribir)
AD-03	Render para app y base	Plan gratis, deploy automático desde GitHub, HTTPS incluido, base cerca de la app	Supabase solo para la base (más latencia); Heroku (sin plan gratis)
AD-04	Roles con grupos de Django y decorador por vista	Simple, visible en cada vista y fácil de testear	Permisos por modelo de Django
AD-05	Multiempresa con columna `negocio` y middleware	Una sola base y un solo código para todos los negocios	Una base o un esquema por negocio
AD-06	Lógica de negocio en `services.py`	Las vistas quedan finas y el mismo núcleo sirve para caja, pedidos y QR	Lógica repetida en cada vista

AD-07	Carritos en la sesión	No ensucia la base con carritos abandonados	Tabla de carritos
AD-08	Precios y totales congelados	Los reportes y comprobantes viejos no cambian cuando cambia un precio	Calcular siempre con el precio actual
AD-09	Entregar un pedido es cobrarlo	No existen pedidos "entregados" sin venta; el stock y la caja cierran	Botón "marcar entregado" separado del cobro
AD-10	Venta QR pendiente con stock reservado	Nadie vende ese stock mientras el cliente paga; si cancela, vuelve	Crear la venta recién al aprobarse el pago
AD-11	Lo fiado fuera de la caja e ingresos	La caja cuadra con la plata real	Contar el fiado como venta cobrada
AD-12	Auditoría con señales y una función común	Login y logout se registran solos; las acciones sensibles con una línea	Logs en archivo
AD-13	Adjuntos en la base de datos	El disco del hosting gratuito se borra en cada deploy	Disco local o S3 (pago)
AD-14	Degradación sin credenciales	Se puede desarrollar, testear y mostrar sin cuentas reales	Exigir todas las credenciales para arrancar
AD-15	Cambios de datos en producción por migración	Render gratis no tiene consola; quedan versionados y se pueden repetir	Cambios a mano en la base

12. Calidad y pruebas

- **485 tests automatizados** con el framework de tests de Django, organizados por app (`tests.py` y archivos `tests_*.py`).
- Cubren permisos por rol (403), aislamiento entre negocios (404), reglas de venta, stock, fiado, puntos, turno y arqueo, pedidos, cobro QR con Mercado Pago simulado, facturas y pagos, adjuntos, reportes y paginación.
- Las integraciones externas se prueban sin red: Mercado Pago y el mail se reemplazan por dobles de prueba.
- Para que la suite corra rápido, los tests usan un hasher de contraseñas liviano.
- Definición de terminado: historia implementada, tests nuevos en verde, suite completa en verde y cambio publicado en Render.

13. Riesgos técnicos y evolución

Riesgo o límite	Situación actual	Evolución propuesta
Plan gratuito de Render	El sitio duerme sin uso y la base gratuita tiene vencimiento	Plan pago y backups automáticos
Adjuntos en la base	Sirve para pocos archivos chicos	Almacenamiento de objetos (S3 o similar) si crece el volumen

Mercado Pago global	Una sola conexión para toda la instalación	Conexión por negocio
Consultas periódicas	La caja y la pantalla del cliente consultan el estado cada pocos segundos	WebSockets o eventos del servidor
Reservas vencidas	Dejan de contar solas, pero quedan con estado "reservada"	Tarea programada que las marque como "liberadas"
Auditoría	Solo se consulta desde el panel de Django	Pantalla propia con filtros